

Member ID: _____

Time: _____

Rank: _____



C# PROGRAMMING (330)

STATE 2024

PRODUCTION:

C# Programming

_____ (400 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder. The project folder for you has already been provided: State_CSharp
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.
5. You will need to use Visual Studios 2019 or greater to complete this exam.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter. It is HIGHLY recommended that you place all of the files into one folder.

The graders will *not* compile or alter your source code to correct for this.

Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- The goal of this assessment is to create a Windows Form App that will allow the user to generate a list from a text file. The txt file will not need to be adjusted.
- The txt file is designed to be read line by line.
- A Visual Studio project folder with the main project file is provided in the contest folder (SanJuanBasinMining.sln).
- All of the object classes and helper classes have been created for you; however, you will still be required to program certain actions in those classes. In addition, the form design has been already created; however, there might be an instance where you must adjust the properties and actions of the objects on the forms.
- There are no interactive objects on the Form. The form only contains a Text Box called consoleBox.
- The program will read the text file called mineralsText.txt.
- Your program will perform multiple calculations; you will code these:
 - **Mineral Production Cost to Mine:** Calculated in **Minerals.cs** per each mineral imported:
 - **Price * Total Tons.**
 - **Total Operating Cost:** Calculated in **Resources.cs** for sum of all mineral production costs:
 - **Total Production Cost * StateTax**

- **Cost to Ship Per City:** Calculated in **Resources.cs** the cost to ship all minerals per city:
 - **Distance to City * Mileage Fee * Total Tonnage**
- Once the list is displayed, it will stay active on the form until the form is closed.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a goal of good code.

Commenting for Source Code Review (see the rubric):

- Certain sections of your code will be graded. These gradable blocks of code can range from creating data structures, method algorithms, exception handling, and class construction.
- The grading rubric contains a section called Source Code Review: in this section are listed a description all of the graded programming concepts.
- Each gradable item must have a comment listed at its beginning, and the comment must be prefixed with the comment flag. The flag helps the graders easily locate the code to increase the effectiveness of grading.
- The flag will always use this naming convention: **SC#** (NOTE: the # symbol will be replaced with sequential numbering, i.e. **SC1**, **SC2**, **SC3**, etc.
- No explanation in the comment with the flag is required, only the comment flag; however, any information placed in the comment could help the grader better understand and avoid any costly errors.
- The comment flag needs to be placed in close proximity to the block of code it represents.
- If a comment flag is not present, you will not receive full credit.
- In this example the Source Code Review has a gradable section of code for printing to the console (Remember these are non-related examples):
 - SC12: **WriteLine** method in the **Form1** class is printing the correct object _____ 10 pts
 - The user will place the code above the method call:

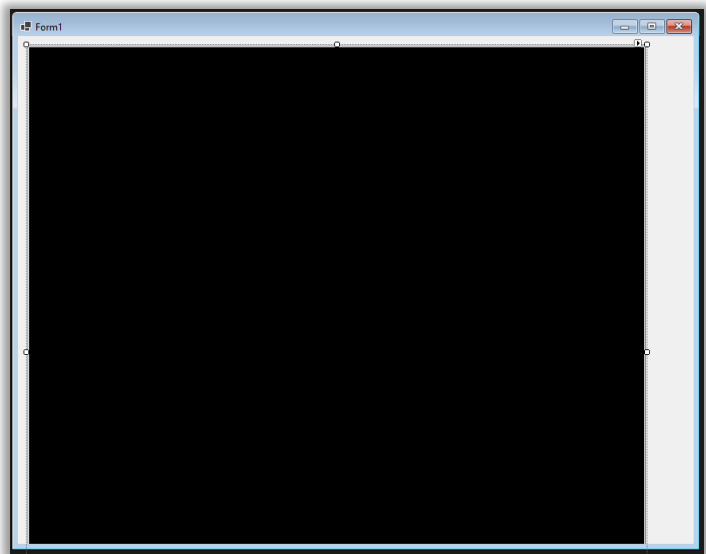
```
//SC12 printing a sarcastic comment  
Console.WriteLine("Get ready for some fun!");
```

State_CSharp

A mining company in New Mexico specializes in extracting minerals that are used to create lithium batteries. The company is wanting you to develop a prototype software to import a list of minerals from another computer system that can only send out text files. Your job will be to read the text file of the various minerals that they mine, and then perform calculations to determine expenditures regarding the production and distribution of these minerals to various cities in North America.

Premade Form

A premade form (see image to the right) has been created for you. It does not need to be changed. The form is called `Form1` and it has one object embedded on it, which is a text box. The text box has its background turned to black to emulate a console output. The name of the object is **`consoleBox`**, but remember, it is NOT a real console screen.



Input

The only input into the program will be performed by the code when it reads the text file that has information about the various minerals. There is NO input that will take place from user interaction via the form.

Text Document

A sample of the text document is provided (see image to the right). Each line represents a different field of information. There are four fields that make up a record. In the example provided there are two complete records. Listed below is the order of the fields:

- Mineral Symbol
- Mineral Name
- Extraction Cost
- Tons Mined

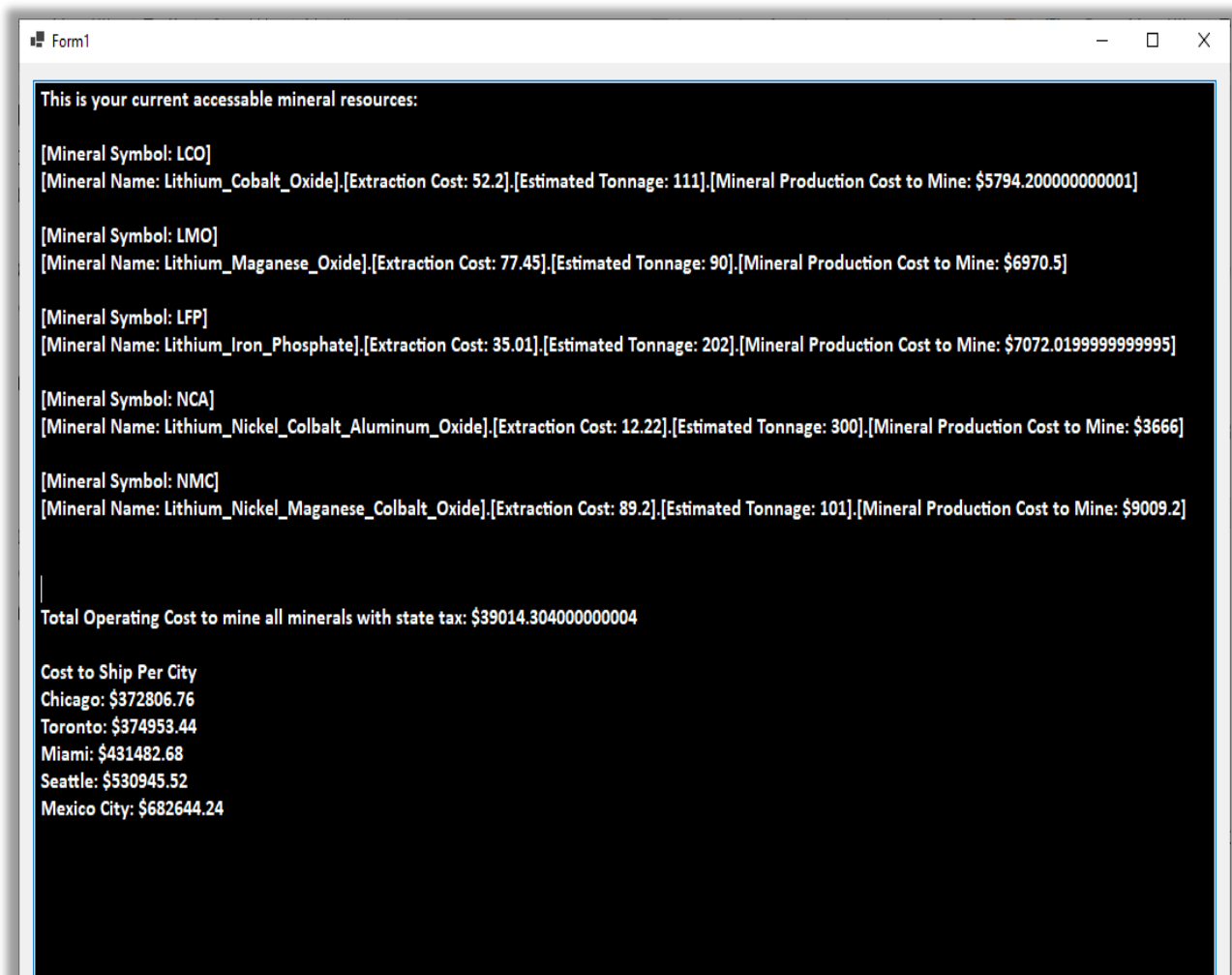
```
NCA
Lithium_Nickel_Colbalt_Aluminum_Oxide
12.22
300
NMC
Lithium_Nickel_Maganese_Colbalt_Oxide
89.20
101
STOP
```

At the bottom of the list is the word “STOP”: which is the termination flag. A termination flag in this case is the word that the program will look for to know when to stop reading the document.

Failure to stop reading could cause undesirable consequences. “STOP” is not part of any record and should not appear in your output.

Output

A sample of the output is provided below. The list of minerals in the center of the program will be extracted from the information contained in the text document. You will be responsible for formatting the information. The bottom section displayed at the bottom of the text box. In addition, you will also be responsible for performing the calculations for the Mineral Production Cost to Mine. The currency values do NOT have to be formatted to US Currency...extra values beyond the hundreths is acceptable; however, your calculations should be exactly as seen in the values below. You will also be responsible for calculating the Total Operating Cost, as well as the Cost to Ship Per City.



```
Form1
-
□
X

This is your current accessible mineral resources:

[Mineral Symbol: LCO]
[Mineral Name: Lithium_Cobalt_Oxide].[Extraction Cost: 52.2].[Estimated Tonnage: 111].[Mineral Production Cost to Mine: $5794.200000000001]

[Mineral Symbol: LMO]
[Mineral Name: Lithium_Maganese_Oxide].[Extraction Cost: 77.45].[Estimated Tonnage: 90].[Mineral Production Cost to Mine: $6970.5]

[Mineral Symbol: LFP]
[Mineral Name: Lithium_Iron_Phosphate].[Extraction Cost: 35.01].[Estimated Tonnage: 202].[Mineral Production Cost to Mine: $7072.0199999999995]

[Mineral Symbol: NCA]
[Mineral Name: Lithium_Nickel_Cobalt_Aluminum_Oxide].[Extraction Cost: 12.22].[Estimated Tonnage: 300].[Mineral Production Cost to Mine: $3666]

[Mineral Symbol: NMC]
[Mineral Name: Lithium_Nickel_Maganese_Cobalt_Oxide].[Extraction Cost: 89.2].[Estimated Tonnage: 101].[Mineral Production Cost to Mine: $9009.2]

|

Total Operating Cost to mine all minerals with state tax: $39014.304000000004

Cost to Ship Per City
Chicago: $372806.76
Toronto: $374953.44
Miami: $431482.68
Seattle: $530945.52
Mexico City: $682644.24
```

Requirements/Explanations

1. You must use the provided Visual Studio project called “**SanJuanBasinMining**” in the `State_CSharp` folder. This project will contain a “**Form1.cs[design]**” that has been already created in the organization and naming conventions.
2. No additional objects can be added to the form design.
3. The following class files have already been provided: **Form1.cs**, **Resources.cs**, **Minerals.cs**, **Cities.cs**, and the text file `mineralsText.txt` is stored in the project folder.
4. Your contestant number must appear as a comment at the top of the **Form1.cs** file.
5. You will be programming methods in **Form1.cs**, methods in the helper class **Resources.cs**, **Minerals.cs**, and **Cities.cs**.
6. The **Form1.cs** class is driver class that will load the form, create the **Resources** helper class object, and most importantly, it will read the text file and send the information to the **Resources** object method designed to create the **Minerals** objects.
7. The **Resources.cs** class is a helper class that is used to perform the calculations, and the creation of **Cities** and **Minerals** and storing objects, and prints to text box object.
8. The **Minerals.cs** object class will create the objects with the information for each mineral from the text document. It will also have a *ToString()* method that will create part of the print to screen information that will be used in the **Resources** object. The **Mineral Production Cost** is also calculated in these objects.
9. The **Cities.cs** object class will store information about the cities where the minerals will be transported. These objects will be created in the **Resources.cs** object. **Cities** is complete and does not need to be altered.
10. Provided Form Objects: **consoleBox**: a text box object that will store the output print.
11. Provided class and method implementation tips:
 - a. **Form1.cs** method *readFile*: this method will be used to read the text file and add **Minerals** objects to a data structure via the **Resources** object method *mineralProcurement()*. The **Minerals** will be stored in a data structure housed in the **Resources** object created in **Form1**.
 - b. **Resources.cs** constructor: creates a resources object (no parameters are needed).
 - c. **Resources.cs** method *setCities()*: use this method to create **Cities** objects to be added to a list. The second parameter in **Cities** objects is randomly generated.
 - d. **Resources.cs** method *mineralProcurement()*: use this method to create **Minerals** objects to be added to a list.
 - e. **Resources.cs** method *getTotalOperationalCost()*: use this method to find the **Total Operating Cost** by using **Minerals** object method *getProductionCost()*.
 - f. **Resources.cs** method *getCostTransport()*: use this method to find the **Cost to Ship Per City** and returns a string that has the cost for every city.
 - g. **Resources.cs** method *getMineralInventory()*: use this method to get a string of all the minerals to print to console.
 - h. **Resources.cs** method *getMineralTonnage()*: use this method to get a sum of integer values for all the minerals.

- i. **Resources.cs** method ***ToString()***: this method prints everything to the text box.
- j. **Form1.cs**, **Minerals.cs** and **Resources.cs**: you will code all methods in these object classes

Solution and Project (NOTE: UC represents uppercase and LC represents lowercase)		
The VS project file is present on the flash drive in a single folder with your contest ID		20 points
Program Execution (If the program does not execute, then the remaining items in this section receive a score of zero)		
When program starts a list of minerals appears.		10 points
Layout matched output example: Mineral Records, followed by Total Operating Cost, finalized with list of Cost to Ship Per City		20 points
Error message appears if the file cannot be found		30 points
All fields for the record are in the proper order		20 points
Records are in the proper order		20 points
Mineral Production Cost to Mine for all records is accurate to the tenths place value		20 points
All currency related values are prefixed with the \$ symbol		10 points
Total Operating Cost to mine all minerals with state tax; accurate to the tenths place value		20 points
Cost to Ship Per City is listed in exact order with different values prefixed with \$ symbol		10 points
Subtotal		/180 Points

Source Code Review <i>NOTE: you must place the comment flag in front of the comment in your code in order to get full credit. The comment flag will precede the explanation. For example, if the flag is SC1, your comment must read as “#SC1...” in front of the part of the code being reviewed. Code must work to get credit.</i>		
A comment containing the contestant number is present at the top of the Form1.cs file		10 points
SC1: Form1.cs class readFile() Place a SC1 comment by these four sections: (1) Code demonstrates reading of the file (2) Code that recognizes the word STOP; (3) Code that handles the construction of the objects by passing to Resources object method mineralProcurement() ; (4) Try/Catch exception handlers for missing file.		40 points
SC2: Resources.cs class setCities() code creates and add Minerals objects to a data structures; mileage is randomly created.		20 points
SC3: Resources.cs class mineralProcurement() code creates and add Cities objects to a data structures		10 points
SC4: Resources.cs class getTotalOperationalCost() code gets production costs from all minerals		10 points
SC5: Resources.cs class getCostTransport() code calculates the cost to transport cargo to all cities; all city information must come from Cities objects; the return object must have all of the printing format per the output example for all cities from one method call		30 points
SC6: Resources.cs class getMineralInventory() code generates the printable object of all minerals objects read from file.		20 points
SC7: Resources.cs class getMineralTonnage() code calculates the total amount of tons for all of the minerals		20 points
SC8: Resources.cs class ToString() code create the out to print to the text box. All information in string must come from helper methods		20 points
SC9: Minerals.cs class constructor: properly handles four parameters as arguments (String, String, double, int).		10 points
SC10: Minerals.cs class getProductionCost() : properly calculates production cost based upon formula.		10 points
SC11: Minerals.cs class ToString() : properly formats all data for each Mineral object to be printed to console.		10 points
SC12: Minerals.cs class printMineral() : is used to call ToString() for all print functions.		10 points
Subtotal		/220 Points
Total Points		/400 Points